

Structures discrètes et algorithmes

Corrigé

Exercice 1 (4 points)

On dispose d'une liste triée L contenant n entiers ; la structure de données adoptée pour coder L est précisée dans les questions ci-dessous. On souhaite y insérer k entiers figurant dans un tableau T en conservant l'ordre des $n + k$ entiers. Décrire un algorithme permettant d'atteindre cet objectif en $O(n + k \log k)$ dans chacun des deux cas suivants :

1. la liste L est donnée sous la forme d'un tableau possédant au moins $n + k$ cases ;
2. la liste L est donnée sous la forme d'une liste chaînée à laquelle on accède par un pointeur indiquant l'adresse du premier élément de L .

Dans les deux cas, on indiquera pourquoi la complexité est bien en $O(n + k \log k)$.

Corrigé

1. On commence par trier le tableau T en utilisant le tri tas ; la complexité de cette opération est en $O(k \log k)$.

Principe : on fusionne les tableaux L et T en commençant par la fin de chacun de ces tableaux et en se déplaçant dans ces tableaux de droite à gauche ; on copie le résultat de la fusion dans le tableau L à partir du plus grand indice de ce tableau.

Pseudo-code : on suppose que les tableaux sont indicés à partir de 0.

On utilise trois entiers : i, j et h .

```

 $i \leftarrow n - 1;$ 
 $j \leftarrow k - 1;$ 
 $h \leftarrow n + k - 1;$ 
tant que  $i \geq 0$  et  $j \geq 0$  faire
    si  $L[i] \geq T[j]$  alors
         $L[h] \leftarrow L[i];$ 
         $i \leftarrow i - 1;$ 
    sinon
         $L[h] \leftarrow T[j];$ 
         $j \leftarrow j - 1;$ 
     $h \leftarrow h - 1;$ 
tant que  $i > 0$  faire
     $L[h] \leftarrow L[i];$ 
     $i \leftarrow i - 1;$ 
     $h \leftarrow h - 1;$ 
tant que  $j > 0$  faire
     $L[h] \leftarrow T[j];$ 
     $j \leftarrow j - 1;$ 
     $h \leftarrow h - 1;$ 

```

Cette fusion de L et T a une complexité de l'ordre de $n + k$; en tout, la complexité est en $O(n + k + k \log k)$, c'est-à-dire en $O(n + k \log k)$.

2. On commence par trier le tableau T en utilisant le tri tas ; la complexité de cette opération est en $O(k \log k)$.

Principe : on fusionne la liste chaînée L et le tableau T en partant du début de L et du premier indice de T .

Pseudo-code : on suppose que le tableau T est indicé à partir de 0 et que la liste L est constituée de maillons comportant deux champs, appelées *donnée* et *suivant* : *donnée* contient un entier, *suivant* contient l'adresse du maillon suivant dans L .

On utilise une fonction *créer_maillon* qui permet de créer un nouveau maillon à insérer dans L . Cette fonction admet deux arguments : une donnée entière à placer dans le maillon, une adresse de maillon permettant d'effectuer le chaînage.

La valeur *rien* représente un pointeur vers rien.

On suppose pour simplifier que L n'est pas vide.

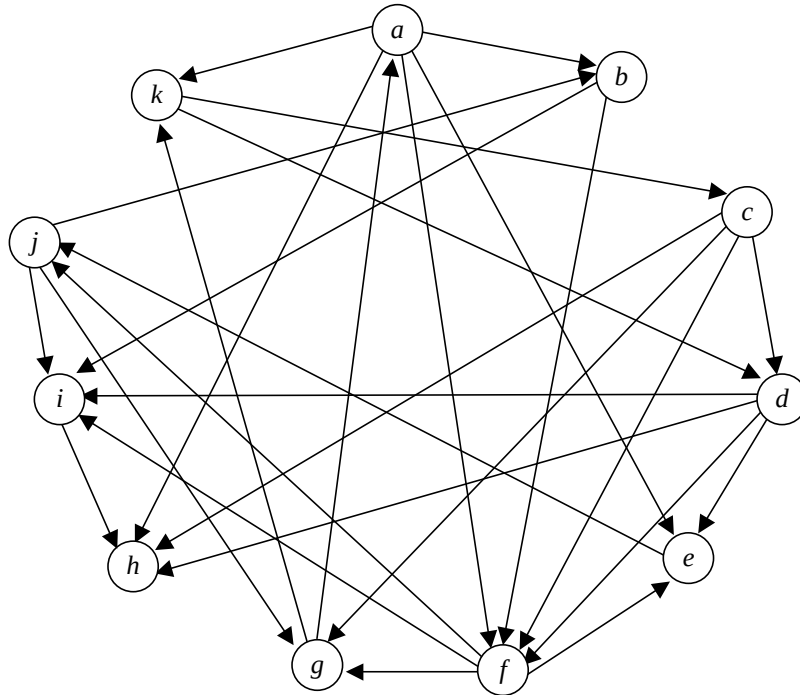
On utilise un entier j et deux pointeurs p et q .

```
 $j \leftarrow 0;$ 
 $p \leftarrow \text{début de } L;$ 
 $q \leftarrow p.\text{suivant};$ 
  tant que  $T[j] < p.\text{donnée}$ , faire           //on ajoute les nouveaux entiers en début de liste quand
                                           //ces entiers sont plus petits que le plus petit entier de  $L$ 
    début de  $L \leftarrow \text{créer\_maillon}(T[j], \text{début de } L);$ 
     $j \leftarrow j + 1;$ 
tant que  $j < k$  faire
  si  $q = \text{rien}$  ou  $q.\text{donnée} > T[j]$  faire
     $p.\text{suivant} \leftarrow \text{créer\_maillon}(T[j], q);$ 
     $p \leftarrow p.\text{suivant};$ 
     $j \leftarrow j + 1;$ 
  sinon
     $p \leftarrow q;$ 
     $q \leftarrow p.\text{suivant};$ 
```

Cette fusion de L et T a une complexité de l'ordre de $n + k$; en tout, la complexité est en $O(n + k + k \log k)$, c'est-à-dire en $O(n + k \log k)$.

Exercice 2 (6 points)

On considère le graphe orienté G ci-dessous.



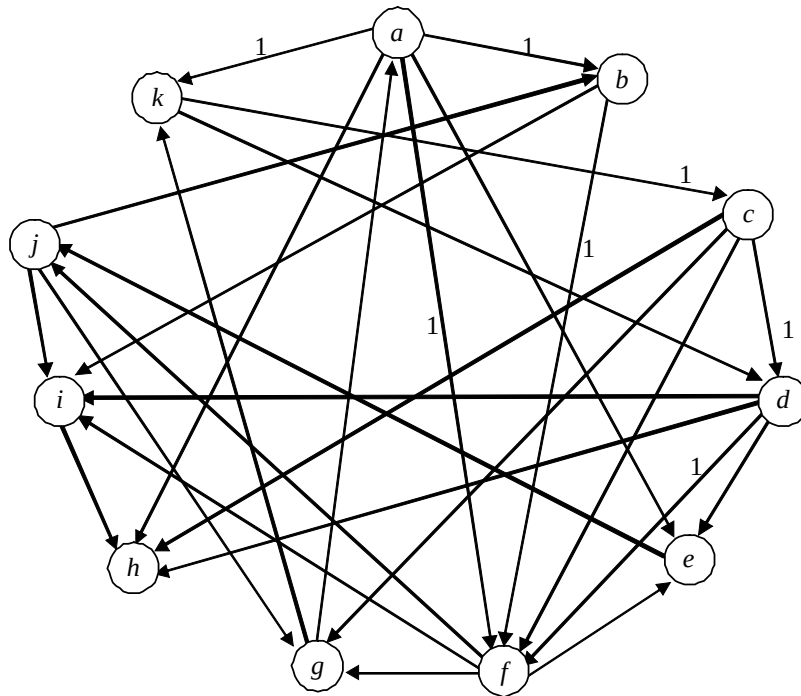
1. On cherche dans G le nombre maximum de chemins deux à deux arc-disjoints de a à f . On propose les trois chemins suivants : (a, f) , (a, b, f) , (a, k, c, d, f) . Cet ensemble est-il un ensemble de chemins deux à deux arc-disjoints de a à f de cardinal maximum ? Si oui, le prouver à l'aide d'arguments issus du cours. Sinon, trouver, en utilisant les trois chemins proposés et un algorithme du cours qu'on précisera, un ensemble de chemins deux à deux arc-disjoints de a à f de cardinal maximum.

Corrigé

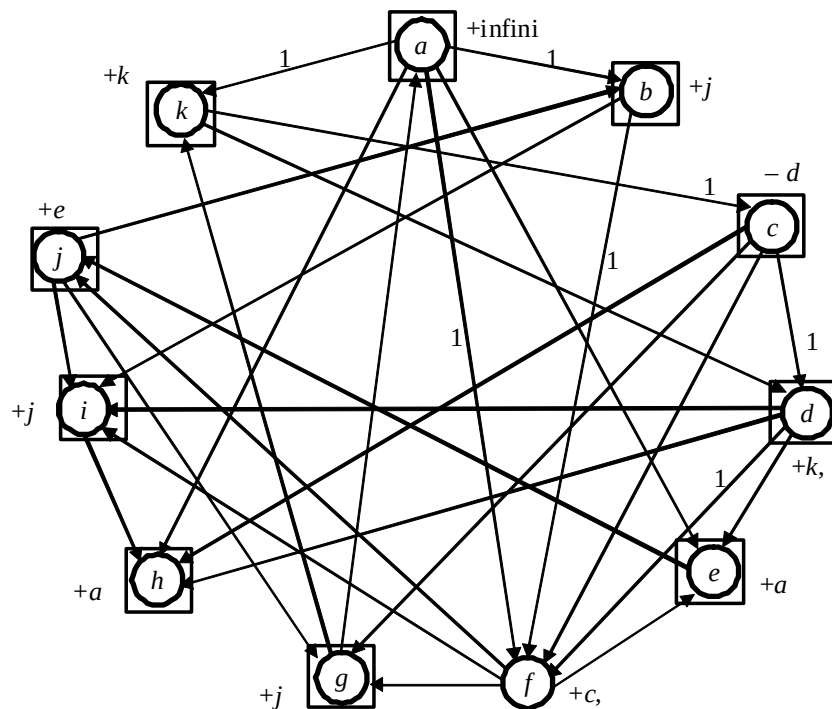
Il s'agit d'une question de cours. Le nombre maximum de chemins arc-disjoints peut être déterminé en modélisant le problème selon un calcul de flot dans un réseau :

- en considérant le réseau formé par le graphe, toutes les capacités égales à 1, la source : a , le puits : f ;
- en calculant le flot de valeur maximum dans ce réseau.

Démarrer avec les chemins indiqués revient à démarrer avec les flux représentés ci-dessous avec la convention que les arcs n'indiquant pas de flux ont un flux nul.



On applique l'algorithme de Ford et Fulkerson en cherchant une chaîne augmentante.
On marque le sommet a avec $+\infty$, aucun autre sommet n'est marqué, aucun sommet n'est examiné.

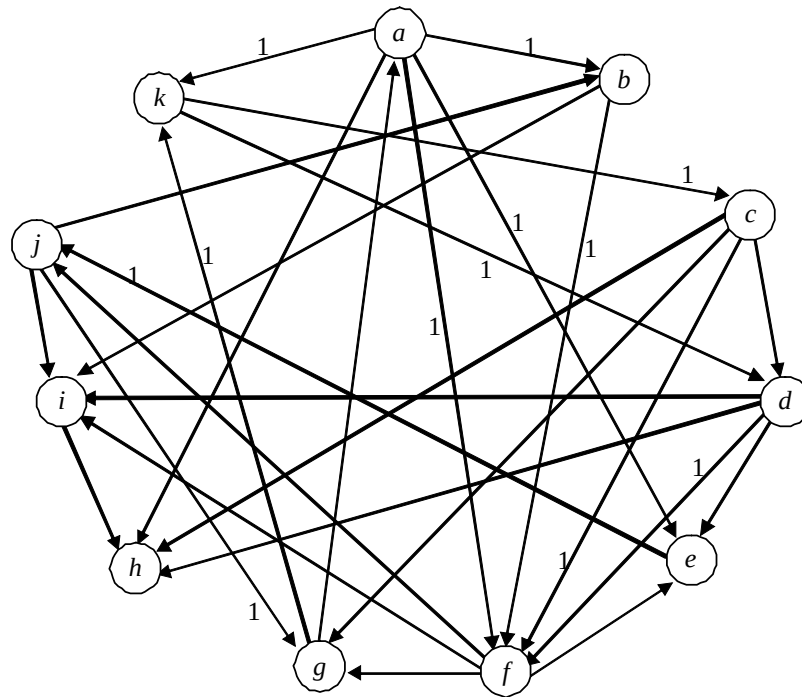


On examine le sommet a : on marque les sommets e avec $(a, +1)$ et h avec $(a, +1)$.
On examine le sommet e : on marque le sommet j avec $(e, +1)$.
On examine le sommet h : on ne marque rien.
On examine le sommet j : on marque les sommets b avec $(j, +1)$, g avec $(j, +1)$ et i avec $(j, +1)$.
On examine le sommet b : on ne marque rien.
On examine le sommet g : on marque le sommet k avec $(g, +1)$.
On examine le sommet i : on ne marque rien.

On examine le sommet k : on marque le sommet d avec $(k, +1)$.
 On examine le sommet d : on marque le sommet c avec $(d, -1)$.
 On examine le sommet c : on marque le sommet f avec $(c, +1)$.

Le sommet f est marqué : on a trouvé une chaîne augmentante, la chaîne $a \rightarrow e \rightarrow j \rightarrow g \rightarrow k \rightarrow d \leftarrow c \rightarrow f$.

On utilise cette chaîne pour augmenter la valeur du flot ; on obtient les flux suivants :



En recommençant la recherche d'une chaîne augmentante, on marque le sommet a puis, en examinant a le sommet h . En examinant h , on ne marque aucun sommet. Tous les sommets marqués ont été examinés et f n'est pas marqué : le flot est de valeur maximum.

Pour confirmer cette conclusion, on peut remarquer que la coupe $(M, \overline{M})$ avec $M = \{a, h\}$, obtenue en regroupant les sommets marqués lors de la dernière application de l'algorithme de marquage, contient les arcs (a, b) , (a, e) , (a, f) et (a, k) et est donc de capacité 4, valeur du flot déterminé.

En suivant des chemins portant des flux de 1 et en ne prenant pas deux fois le même arc, on obtient soit les chemins :

(a, f) , (a, b, f) , (a, k, d, f) , (a, e, j, g, k, c, f)

soit les chemins

(a, f) , (a, b, f) , (a, k, c, f) , (a, e, j, g, k, d, f) .

Ces chemins forment un ensemble de chemins deux à deux arc-disjoints de a à f de cardinal maximum.

2. On considère les deux ensembles disjoints de sommets $A = \{a, b, c\}$ et $B = \{f, g\}$. On cherche le nombre maximum de chemins deux à deux arc-disjoints dans G dont l'origine soit dans A et l'extrémité dans B . Décrire et justifier une méthode pour résoudre ce problème (on ne demande pas de résoudre le problème ainsi modélisé).

Corrigé

On ajoute deux sommets notés s et p .

On relie s à a , b et c .

On relie f et g à p .

Sur tous les arcs ajoutés, on met une capacité entière suffisamment grande pour que ces arcs ne soient pas bloquant. Par exemple, on peut mettre une capacité infinie sur ces arcs ; ou bien : sur les arcs (s, x) avec x qui est a , b ou c , on met une capacité égale au degré extérieur de x , sur les arcs (y, p) avec y qui est f ou g , on met une capacité égale au degré intérieur de y .

On cherche alors un flot de valeur maximum et à flux entiers dans le réseau obtenu en considérant que la source est s et le puits p . On trouve comme précédemment un ensemble de chemins de s à p arc-disjoints sauf en ce qui concerne les arcs d'origine s ou les arcs d'extrémités p . Pour cela, on n'élimine pas les arcs d'origine s ou les arcs d'extrémités p lorsqu'on les a déjà mis dans un chemin de s à p . Plus précisément, il existe alors une bijection entre, d'une part, l'ensemble des chemins de s à p arc-disjoints sauf pour les arcs incidents à s ou p et, d'autre part, les flots à flux entiers. De plus, cette bijection est telle que le nombre de chemins ainsi obtenus est égal à la valeur du flot qui lui est associé. Ainsi, grâce au choix des capacités, l'ensemble associé à un flot de valeur maximum et à flux entiers est de cardinal maximum parmi les ensembles de chemins de s à p arc-disjoints sauf en ce qui concerne les arcs d'origine s ou les arcs d'extrémités p (la preuve se fait comme pour le résultat du cours utilisé dans la question 1). La fin de la justification de cette modélisation vient du fait qu'il existe une bijection conservant les cardinaux entre les ensembles de chemins deux à deux arc-disjoints de A à B et les ensembles de chemins de s à p dans le graphe complété, arc-disjoints sauf en ce qui concerne les arcs d'origine s ou les arcs d'extrémités p .

Les chemins recherchés sont donc obtenus en supprimant des chemins précédents les arcs d'origine s ou les arcs d'extrémités p .

Exercice 3 (10 points)

Soit $G = (X, A)$ un graphe orienté dont les arcs sont valués par une fonction δ à valeurs positives ; pour tout arc a de A , $\delta(a)$ s'appelle le *débit* de a . Soit r un sommet de G . On suppose qu'il existe dans G au moins un chemin de r vers tout autre sommet de G .

Soit C un chemin ; on appelle *débit de C* et on note $\delta(C)$ le minimum des débits des arcs de C : $\delta(C) = \min_{a \text{ arc de } C} \delta(a)$. Autrement dit, le débit d'un chemin est donné par le plus

petit débit des arcs constituant ce chemin. Pour chaque sommet x de G autre que r , on cherche à déterminer un chemin C_x^* de r à x de débit maximum :

$$\delta(C_x^*) = \max_{C_x \text{ chemin de } r \text{ à } x} \delta(C_x).$$

1. (1 point) Déterminer C_d^* dans le graphe G_0 ci-dessous. Justifier la réponse.

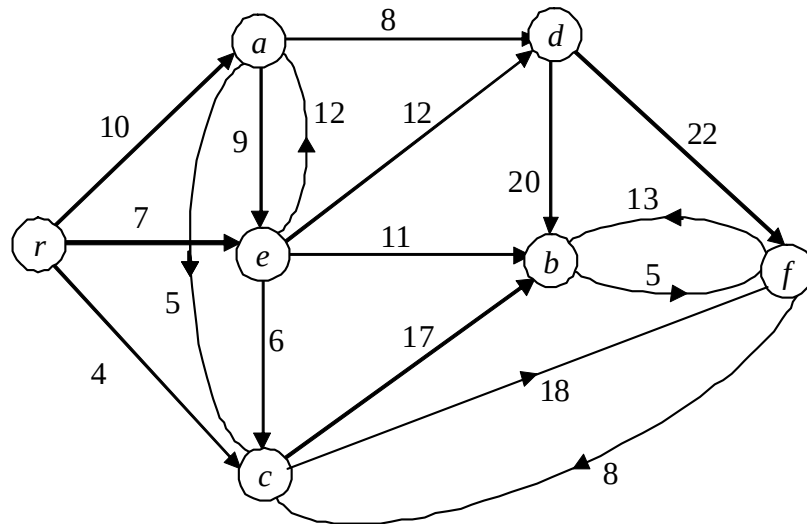
Corrigé

De façon générale, on peut se contenter de chercher le débit maximum en examinant les chemins élémentaires. En effet, si on considère un chemin C passant deux fois par un même sommet x , en enlevant de C le circuit délimité par deux occurrences de x , on obtient un chemin dont le débit n'est pas plus petit que celui de C . En itérant cette opération, on obtient finalement un chemin élémentaire de débit au moins égal à celui du chemin de départ.

Les chemins élémentaires de a à d sont :

- le chemin *rad* de débit 8,
- le chemin *raed* de débit 9,
- le chemin *red* de débit 7,
- le chemin *read* de débit 7.

C_d^* est donc le chemin *raed* de débit 9.



2. (3 points) Adapter l'algorithme de Dijkstra pour que celui-ci détermine, pour tout sommet x de G , un chemin C_x^* de r à x de débit maximum ; on décrira précisément cette adaptation ; en revanche, il n'est pas demandé de la justifier.

Corrigé

On peut adapter l'algorithme de Dijkstra ainsi, en notant n l'ordre de G :

- $A \leftarrow \{r\}$
- $pivot \leftarrow r$
- $\pi(r) \leftarrow \infty$
- pour tout sommet x différent de r , faire $\pi(x) \leftarrow -\infty$
- pour j variant de 1 à $n - 1$, faire
 - * pour tout sommet y non encore dans A et successeur de $pivot$, faire
 - si $\min(\pi(pivot), \delta(pivot, y)) > \pi(y)$, alors
 - $\pi(y) \leftarrow \min(\pi(pivot), \delta(pivot, y))$
 - $père(y) \leftarrow pivot$
 - * chercher, parmi les sommets non dans A , un sommet y tel que $\pi(y)$ soit maximum
 - * $pivot \leftarrow y$
 - * $A \leftarrow A \cup \{pivot\}$

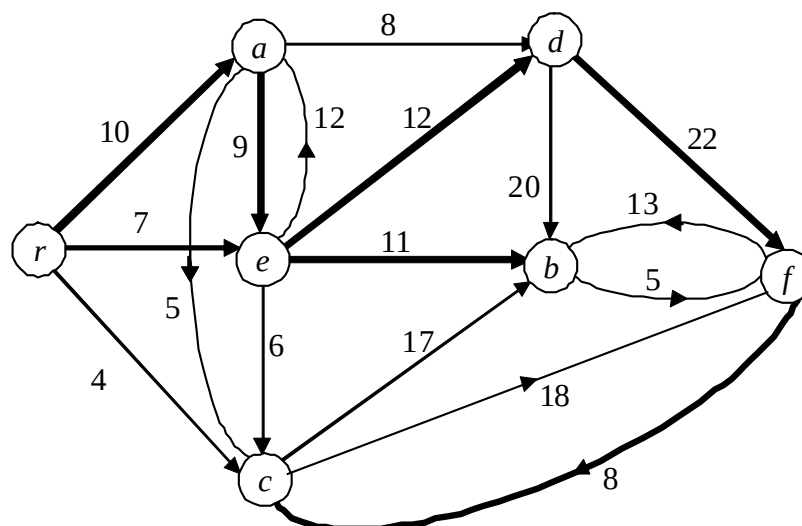
3. (3 points) Appliquer, de façon détaillée, cette adaptation au graphe G_0 à partir du sommet r . On dessinera l'arborescence des chemins obtenus.

Corrigé

On construit un tableau sur le modèle utilisé pour illustrer le déroulement de l'algorithme de Dijkstra.

$\pi, \text{préd}$	pivot	r	a	e	b	d	f	c
Initialisation		∞	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
Etape 1	r		10, r	7, r	$-\infty$	$-\infty$	$-\infty$	4, r
Etape 2	a			9, a	$-\infty$	8, a	$-\infty$	5, a
Etape 3	e				9, e	9, e	$-\infty$	6, e
Etape 4	b					9, e	5, b	6, e
Etape 5	d						9, d	6, e
Etape 6	f							8, f

L'arborescence obtenue est indiquée ci-dessous :



4. (0,5 point) Déterminer la complexité de l'algorithme conçu à la question 2. On justifiera la réponse.

Corrigé

L'algorithme a la même complexité que l'algorithme de Dijkstra. Il est en $O(n^2)$.

5. (1,5 points) La détermination de C_x^* , pour x donné, correspond au problème d'optimisation suivant :

Nom : Chemin de débit maximum (CDM)

Données : un graphe $G = (X, A)$ orienté dont chaque arc a est valué par un débit $\delta(a)$ positif ; deux sommets r et x de G ;

Objectif : déterminer le débit maximum d'un chemin C_x^* de r vers x .

Écrire le problème de décision associé à CDM. Ce problème appartient-il à la classe P ? à la classe NP ? On justifiera les réponses.

Corrigé

Problème de décision

Données : un graphe $G = (X, A)$ orienté dont chaque arc a est valué par un débit $\delta(a)$ positif ; deux sommets r et x de G ; un entier k ;

Question : existe-t-il dans G un chemin de r et x de débit supérieur ou égal à k ?

Ce problème peut être résolu en $O(n^2)$. Comme la taille T de l'instance est au moins égal à n^2 , la résolution du problème se fait en $O(T)$: le problème est dans la classe P.

Comme P est inclus dans NP, le problème est aussi dans la classe NP.

6. (1 point) On suppose maintenant que les valuations $\delta(a)$ des arcs a ne sont plus nécessairement positives (quitte à ne plus les interpréter comme des débits). On pose encore, pour tout chemin C : $\delta(C) = \min_{a \text{ arc de } C} \delta(a)$ et on s'intéresse, pour tout sommet x , à la détermination d'un chemin C_x^* de r vers x vérifiant $\delta(C_x^*) = \max_{C_x \text{ chemin de } r \text{ à } x} \delta(C_x)$, comme plus haut. L'adaptation proposée à la question 2 s'applique-t-elle encore quand on ne suppose plus les valuations $\delta(a)$ positives ? On justifiera la réponse.

Corrigé

Si on ajoute une même quantité Q à tous les débits des arcs, les débits de tous les chemins sont augmentés de Q ; cette opération ne modifie donc pas les classements des chemins par rapport au débit. Si des valuations sont négatives, on peut ajouter une quantité Q suffisamment grande à tous les débits des arcs pour ne plus avoir que des débits positifs puis appliquer l'adaptation au graphe valué obtenu. Il suffit alors, après avoir fait tourner l'algorithme, de retirer Q à tous les débits des chemins pour avoir l'arborescence et les débits des chemins. On constate en fait, en examinant l'algorithme, qu'ajouter Q puis le retirer est inutile. L'adaptation peut donc s'appliquer directement au cas de valuations de signe quelconque.

Une autre façon de voir est de constater que la preuve de l'algorithme se fonde sur le fait que, le long d'un chemin issu de r , le débit ne fait que décroître. De même d'ailleurs que la preuve de l'algorithme de Dijkstra se fonde sur le fait que, lorsque les longueurs des arcs sont positives ou nulles, le long d'un chemin issu de r , les longueurs ne font que croître. On pourrait ainsi montrer directement, sans passer par l'étape consistant à translater les valuations, que l'adaptation proposée plus haut convient à des valuations de signe quelconque.